# TEC004 — Advanced Computer Programming

## Sample Industry Project Proposals

*5 Project Templates Covering Full Syllabus (OOP, Web Scraping, Database, Data Analysis, AI)*

| | |
|---|---|
| **Project ID** | TEC004/01 |
| **Project Title** | Real Estate Price Tracker & Market Analyzer |
| **Mentor** | |
| **Project Description** | |
| Build an end-to-end Python application that automatically collects real estate listing data from public property websites, stores it in a structured database, performs market analysis, and generates interactive visualizations for price trends and regional comparisons.<br>**Syllabus Coverage:** This project integrates OOP design (classes for Property, Scraper, Analyzer), web scraping with BeautifulSoup and Selenium (dynamic listing pages), JSON/CSV file processing, SQLite database for persistent storage, Pandas for data cleaning and analysis, MatPlotLib for visualization, and multi-threading for parallel scraping.<br>**Expected Deliverables:**<br>SP1 - OOP-based scraper system: Property class hierarchy (Apartment, House, Land inheriting from base Property), Scraper class with abstract methods, DatabaseManager class with CRUD operations.<br>SP2 - Web scraping module: BeautifulSoup parser for static pages + Selenium WebDriver for JavaScript-rendered listings. Rate-limited, robots.txt-compliant crawling with retry logic.<br>SP3 - SQLite database: Normalized schema storing property details (price, area, location, amenities, listing date). Support for historical price tracking across multiple scraping sessions.<br>SP4 - Data analysis pipeline: Pandas-based cleaning (handling missing values, outlier detection, price normalization), statistical analysis (average price per district, price-per-m2 trends, seasonal patterns).<br>SP5 - Visualization dashboard: MatPlotLib charts including price distribution histograms, time-series trend lines, geographic heatmaps, and comparative bar charts by district/property type.<br>SP6 - Technical report: System architecture, data pipeline documentation, analysis findings, and recommendations for real estate investment based on collected data. | |
| **Project Skills** | |
| x | Object-Oriented Programming (Classes, Inheritance, Abstract Classes) |
| x | Web Scraping (BeautifulSoup, Selenium, requests) |
| x | Database Management (SQLite, SQL queries) |
| x | Data Analysis (Pandas, NumPy) |
| x | Data Visualization (MatPlotLib) |
| x | Multi-threading (concurrent scraping) |
| x | File I/O (JSON, CSV processing) |
| **Research Component** | |

**(Phase 1 - Foundation & Data Collection):**

- Design OOP class hierarchy: Property (abstract base), Apartment, House, Land with encapsulation and polymorphism.
- Implement web scraper using requests + BeautifulSoup for static property listing pages.
- Add Selenium WebDriver for sites with dynamic JavaScript content (infinite scroll, lazy loading).
- Implement multi-threaded scraping to crawl multiple districts concurrently.
- Design and create SQLite database schema; implement CRUD operations via Python.
- Export raw data to JSON and CSV formats for backup and portability.

**(Phase 2 - Analysis & Visualization):**

- Clean and preprocess data using Pandas (handle duplicates, missing values, normalize prices).
- Perform statistical analysis: price trends by district, property type distribution, price-per-m2 rankings.
- Build MatPlotLib visualizations: histograms, scatter plots, time-series charts, heatmaps.
- Create automated reporting pipeline that generates weekly market summary reports.
- Submit Midterm Report with data collection results and preliminary analysis.

**(Phase 3 - Integration & Presentation):**

- Integrate all modules into a unified command-line application.
- Add advanced features: price prediction using historical trends, alert system for price drops.
- Comprehensive testing, error handling, and documentation.
- Final presentation with live demo of scraping, database queries, and visualization dashboard.

| | |
|---|---|
| **Project ID** | TEC004/02 |
| **Project Title** | Multilingual News Aggregator & Sentiment Analysis Dashboard |
| **Mentor** | |

**Project Description**

Develop a Python-based news aggregation system that crawls Vietnamese and English news websites, processes articles using NLP techniques, stores them in a database, and provides sentiment analysis with interactive visualizations to track public opinion trends on specific topics.

**Syllabus Coverage:** Covers OOP (NewsArticle, Crawler, Analyzer class hierarchy), advanced functions (lambda/map/filter for text processing), web scraping (BeautifulSoup for article extraction), Selenium (for dynamic news sites), JSON/CSV handling, SQLite storage, Pandas analysis, MatPlotLib visualization, multi-threading (parallel crawling), and AI application (sentiment analysis API).

**Expected Deliverables:**

SP1 - News crawler engine: OOP-designed crawler supporting multiple news sources (VnExpress, Tuoi Tre, BBC, Reuters). Abstract base Crawler class with site-specific implementations using inheritance.

SP2 - Text processing pipeline: Advanced functions (lambda, map, filter) for text cleaning, tokenization, and keyword extraction. Decorator-based logging for processing steps.

SP3 - SQLite database: Articles table (title, content, source, date, category, sentiment_score), Keywords table, Sources table with foreign key relationships.

SP4 - Sentiment analysis module: Integration with AI API (e.g., TextBlob, or Generative AI API) to classify article sentiment (positive/neutral/negative).

SP5 - Analytics dashboard: MatPlotLib visualizations showing sentiment trends over time, topic frequency word clouds, source comparison charts, and category breakdowns.

SP6 - Automated daily report: CSV/JSON export of daily news summary with sentiment scores.

**Project Skills**

| | |
|---|---|
| x | Object-Oriented Programming (Classes, Inheritance, Polymorphism) |
| x | Advanced Functions (Lambda, Map, Filter, Decorators) |
| x | Web Scraping (BeautifulSoup, Selenium) |
| x | Database Management (SQLite) |
| x | Data Analysis (Pandas) |
| x | Data Visualization (MatPlotLib) |
| x | AI Application (Sentiment Analysis API) |
| x | Multi-threading (Parallel crawling) |

**Research Component**

**(Phase 1 - Crawling & Storage):**

- Design class hierarchy: BaseCrawler (abstract) -> VnExpressCrawler, BBCCrawler with method overriding.

- Implement BeautifulSoup parsers for each news source; add Selenium for JavaScript-heavy sites.

- Use advanced functions: lambda for text filtering, map for batch processing, decorators for timing/logging.

- Create SQLite database with normalized schema; implement file export (JSON/CSV).

- Implement multi-threaded crawling for concurrent source processing.

**(Phase 2 - Analysis & AI Integration):**

- Build Pandas pipeline for data cleaning, deduplication, and time-series analysis.

- Integrate sentiment analysis using AI API; store scores in database.

- Create MatPlotLib visualizations: sentiment timeline, source comparison, topic heatmap.

- Develop automated daily report generation.

- Submit Midterm Report.

**(Phase 3 - Dashboard & Presentation):**

- Build comprehensive analytics dashboard combining all visualizations.

- Add topic tracking and trend detection features.

- System testing, optimization, and documentation.

- Final presentation with live demo.

| | |
|---|---|
| **Project ID** | TEC004/03 |
| **Project Title** | Job Market Intelligence System for IT Industry in Vietnam |
| **Mentor** | |
| **Project Description** | |

Create a comprehensive job market analysis tool that scrapes IT job postings from Vietnamese recruitment platforms, extracts skill requirements and salary data, stores in a database, and produces actionable insights through data analysis and visualization to help students understand industry demands.

**Syllabus Coverage:** Covers OOP (Job, Company, SkillRequirement classes with inheritance), web scraping (BeautifulSoup for listing pages, Selenium for detail pages with login/pagination), JSON/CSV file management, SQLite database, Pandas for salary and skill analysis, MatPlotLib for market trend visualization, advanced functions (lambda/filter for data processing), and multi-threading.

**Expected Deliverables:**

SP1 - Job scraper: Multi-site scraper for TopCV, VietnamWorks, ITviec using OOP design. Abstract JobScraper class with site-specific implementations. Selenium for paginated results and detail page navigation.

SP2 - Data extraction engine: Parse job titles, companies, salaries, required skills, experience levels, and locations. Use regex and string methods for structured extraction from unstructured HTML.

SP3 - SQLite database: Jobs, Companies, Skills, JobSkills (many-to-many) tables. SQL queries for complex analysis (most demanded skills, salary ranges by experience).

SP4 - Market analysis report: Pandas-based analysis of top skills, salary distributions, experience requirements, geographic distribution of IT jobs.

SP5 - Visualization suite: MatPlotLib charts showing skill demand rankings, salary box plots, job posting trends over time, geographic distribution maps.

SP6 - Career recommendation module: Based on collected data, suggest which skills to learn based on market demand and salary potential.

| **Project Skills** | |
|---|---|
| x | Object-Oriented Programming (Classes, Inheritance, Abstract Classes) |
| x | Web Scraping (BeautifulSoup, Selenium WebDriver) |
| x | Database Management (SQLite, complex SQL queries) |
| x | Data Analysis (Pandas, statistical analysis) |
| x | Data Visualization (MatPlotLib) |
| x | File I/O (JSON, CSV export/import) |
| x | Advanced Functions (Lambda, Filter, Map) |
| x | String Processing (Regex, text extraction) |

**Research Component**

**(Phase 1 - Scraping Infrastructure):**

- Design OOP class hierarchy: BaseJobScraper (abstract), TopCVScraper, ITviecScraper with polymorphism.
- Implement BeautifulSoup parsing for job listing pages; extract structured data from HTML tables.
- Add Selenium for sites requiring JavaScript rendering, pagination handling, and detail page navigation.
- Create SQLite database with normalized schema (Jobs, Companies, Skills, JobSkills).
- Export collected data to JSON and CSV for backup.

**(Phase 2 - Analysis & Insights):**

- Clean data with Pandas: handle salary ranges, normalize skill names, remove duplicates.
- Perform statistical analysis: top 20 most demanded skills, median salary by role, experience distribution.
- Build MatPlotLib visualizations: bar charts, box plots, pie charts, trend lines.
- Generate career recommendation based on skill demand vs. salary data.
- Submit Midterm Report with market analysis findings.

**(Phase 3 - Integration & Reporting):**

- Build automated weekly report generation pipeline.
- Create comparison module: student skills vs. market demand gap analysis.
- Comprehensive testing, documentation, and user guide.
- Final presentation with market insights and career recommendations.

| | |
|---|---|
| **Project ID** | TEC004/04 |
| **Project Title** | PTT Forum Trend Analyzer & Discussion Mining System |
| **Mentor** | |

**Project Description**

Build a Python application that crawls PTT (the largest BBS forum in Taiwan), extracts discussion threads and comments, stores them in a database, performs trend analysis and keyword mining, and visualizes discussion patterns to understand public opinion on technology topics.

**Syllabus Coverage:** Directly covers the syllabus PTT Crawler topic, plus OOP design, BeautifulSoup and Selenium for scraping, JSON/CSV processing, SQLite database, Pandas for text and time-series analysis, MatPlotLib for visualization, advanced functions (decorators for rate limiting, lambda for filtering), multi-threading, and file management.

**Expected Deliverables:**

SP1 - PTT Crawler: OOP-based crawler that navigates PTT boards (Gossiping, Tech_Job, Soft_Job), handles pagination (over-18 verification), extracts post titles, authors, dates, content, and push/boo counts.

SP2 - Data processing pipeline: Use advanced functions (map/filter/lambda) to clean posts, extract keywords, calculate sentiment from push/boo ratios. Decorator-based caching and rate limiting.

SP3 - SQLite database: Posts, Comments, Keywords, Boards tables. Full-text search capability. Historical data tracking for trend analysis.

SP4 - Trend analysis module: Pandas-based time-series analysis of posting frequency, hot topic detection, keyword co-occurrence analysis, author activity patterns.

SP5 - Visualization dashboard: MatPlotLib charts including posting activity heatmaps (hour x day), keyword trend lines, sentiment distribution, top poster rankings, board comparison charts.

SP6 - Technical report with findings on technology discussion trends in PTT.

**Project Skills**

| | |
|---|---|
| x | Web Scraping (BeautifulSoup, requests, PTT-specific techniques) |
| x | Object-Oriented Programming (Classes, Inheritance) |
| x | Advanced Functions (Lambda, Map, Filter, Decorators) |
| x | Database Management (SQLite, full-text search) |
| x | Data Analysis (Pandas, time-series analysis) |
| x | Data Visualization (MatPlotLib) |
| x | Multi-threading (concurrent board crawling) |
| x | File I/O (JSON, CSV) |

**Research Component**

**(Phase 1 - PTT Crawling):**

- Study PTT HTML structure; handle over-18 cookie verification using requests.Session.
- Design OOP classes: PTTCrawler, Post, Comment, Board with encapsulation.
- Implement BeautifulSoup parser for post listings and individual post content.

- Handle pagination (previous page navigation), extract push/boo/neutral counts.
- Create SQLite database; export raw data to JSON/CSV.
- Implement rate limiting with decorator pattern; multi-threaded board crawling.

**(Phase 2 - Text Mining & Analysis):**

- Build text processing pipeline using lambda/map/filter for keyword extraction.
- Pandas analysis: posting frequency trends, hot topic detection, sentiment calculation from push ratios.
- Time-series analysis: hourly/daily patterns, trend detection for specific keywords.
- MatPlotLib visualizations: activity heatmaps, word frequency charts, sentiment trends.
- Submit Midterm Report with PTT data collection and preliminary analysis.

**(Phase 3 - Dashboard & Insights):**

- Build comprehensive trend dashboard.
- Add real-time monitoring mode for breaking topics.
- Cross-board comparison analysis.
- Final presentation with live crawling demo and trend insights.

| | |
|---|---|
| **Project ID** | TEC004/05 |
| **Project Title** | University Course Performance Tracker & Grade Prediction System |
| **Mentor** | |

**Project Description**

Develop a Python application that manages student academic records, processes grade data from various file formats, stores in a database, performs statistical analysis on academic performance, and uses simple ML to predict final grades based on midterm scores and assignment completion rates.

**Syllabus Coverage:** Comprehensive coverage including OOP (Student, Course, GradeBook class hierarchy with inheritance and abstract classes), advanced functions (lambda for grade calculation, decorators for access control), JSON/CSV file processing (importing/exporting grade sheets), SQLite database (student records, courses, enrollments), Pandas (grade statistics, performance analysis), MatPlotLib (grade distributions, performance trends), multi-threading (batch file processing), and AI application (grade prediction model).

**Expected Deliverables:**

SP1 - OOP grade management system: Abstract Person class -> Student, Instructor. Course class with enrollment management. GradeBook class using polymorphism for different grading schemes (weighted, curved, pass/fail).

SP2 - File I/O module: Import grades from CSV files (midterm, final, assignments). Export reports to JSON. Batch processing of multiple class files using multi-threading.

SP3 - SQLite database: Students, Courses, Enrollments, Grades, Assignments tables with proper foreign keys. Complex SQL queries for GPA calculation, class rankings, at-risk student identification.

SP4 - Analytics engine: Pandas-based analysis including grade distributions, pass/fail rates, correlation between attendance and grades, semester-over-semester trends.

SP5 - Visualization reports: MatPlotLib charts including grade distribution histograms, GPA trend lines, course difficulty comparison, performance radar charts.

SP6 - Grade prediction module: Simple AI model using historical data to predict final grades based on midterm scores, assignment completion rate, and attendance. Alert system for at-risk students.

SP7 - Web-based data entry: Simple Selenium-automated form for data entry testing.

**Project Skills**

| | |
|---|---|
| x | Object-Oriented Programming (Classes, Inheritance, Abstract Classes, Polymorphism) |
| x | Advanced Functions (Lambda, Map, Filter, Reduce, Decorators) |
| x | File I/O (JSON, CSV processing, batch operations) |
| x | Database Management (SQLite, complex SQL queries, GPA calculation) |
| x | Data Analysis (Pandas, statistical analysis, correlation) |
| x | Data Visualization (MatPlotLib, multiple chart types) |
| x | Multi-threading (batch file processing) |
| x | AI Application (grade prediction model) |
| x | Web Automation (Selenium for testing) |

| Research Component |
| --- |
| **(Phase 1 - System Design & Data Management):**<br>- Design OOP hierarchy: Person (abstract) -> Student, Instructor. Course, GradeBook with polymorphism.<br>- Implement grading schemes using inheritance: WeightedGrade, CurvedGrade, PassFailGrade.<br>- Build CSV/JSON import/export module with advanced functions (lambda for grade mapping, map for batch processing).<br>- Create SQLite database with normalized schema; implement CRUD operations.<br>- Add decorator-based access control (admin vs. instructor vs. student views).<br>**(Phase 2 - Analysis & Visualization):**<br>- Build Pandas analysis pipeline: GPA calculation, class rankings, pass/fail rates.<br>- Perform correlation analysis: attendance vs. grades, midterm vs. final performance.<br>- Create MatPlotLib visualizations: histograms, box plots, trend lines, radar charts.<br>- Implement multi-threaded batch processing for importing multiple class files.<br>- Submit Midterm Report with system demo and initial analytics.<br>**(Phase 3 - AI & Final Integration):**<br>- Build grade prediction model using historical data patterns.<br>- Create at-risk student alert system based on prediction results.<br>- Add Selenium-based automated testing for data entry forms.<br>- Comprehensive system testing, documentation, and user manual.<br>- Final presentation with live demo of complete system. |

## Syllabus Coverage Matrix

Each project covers different combinations of syllabus topics. Together, all 5 projects ensure 100% coverage.

| Syllabus Topic | P1 | P2 | P3 | P4 | P5 |
|---|---|---|---|---|---|
| OOP (Classes, Methods, Objects) | ✓ | ✓ | ✓ | ✓ | ✓ |
| Inheritance & Abstract Classes | ✓ | ✓ | ✓ | ✓ | ✓ |
| Advanced Functions (Lambda, Map, Filter) | ✓ | ✓ | ✓ | ✓ | ✓ |
| Decorators & Closures | | ✓ | | ✓ | ✓ |
| JSON & CSV Processing | ✓ | ✓ | ✓ | ✓ | ✓ |
| File Management | ✓ | ✓ | ✓ | ✓ | ✓ |
| Web Scraping (BeautifulSoup) | ✓ | ✓ | ✓ | ✓ | |
| Web Automation (Selenium) | ✓ | ✓ | ✓ | | ✓ |
| Pandas & Data Analysis | ✓ | ✓ | ✓ | ✓ | ✓ |
| SQLite Database | ✓ | ✓ | ✓ | ✓ | ✓ |
| MatPlotLib Visualization | ✓ | ✓ | ✓ | ✓ | ✓ |
| Multi-threading | ✓ | ✓ | | ✓ | ✓ |
| AI Application | | ✓ | | | ✓ |